

## บทที่ 6

# การพัฒนาแอปพลิเคชันบนฝั่งวินโดวส์ 2000

จากบทที่แล้ว ได้กล่าวถึงการทำงานของแอปพลิเคชันทั้งสองเซิร์ฟเวอร์ ในบทนี้จะกล่าวถึงการพัฒนาแอปพลิเคชันบนฝั่งวินโดวส์ 2000 ซึ่งแอปพลิเคชันนี้ไม่จำเป็นต้องทำงานบนแอคทีฟไคลเรกทอรี แต่สามารถทำงานที่เครื่องที่เป็นเซิร์ฟเวอร์เครื่องใดก็ได้ ที่มีระบบปฏิบัติการวินโดวส์ 2000 ติดตั้งอยู่ในเครื่องนั้น

### 6.1 แนวคิดในการออกแบบโปรแกรม

สำหรับแอปพลิเคชันบนฝั่งวินโดวส์ 2000 นี้ จะมีหน้าที่ในการตรวจสอบการเข้าออกของผู้ใช้ จึงมีแนวคิดในการที่จะตรวจสอบผู้ใช้งานในแต่ละเครื่องว่าเป็นผู้ใช้ใด ซึ่งในการตรวจสอบว่าผู้ใช้ใดใช้งานอยู่บนเครื่องนั้น เป็นเรื่องที่ไม่ยากเกินไปนัก แต่การที่จะทราบว่าผู้ใช้ได้เข้าสู่ระบบนั้น เป็นเรื่องที่สำคัญอย่างยิ่ง เนื่องจากเมื่อผู้ใช้ล็อกอินเข้าสู่ระบบ ผู้ใช้ก็ต้องการที่จะใช้งานเครื่องนั้นๆทันที สำหรับการเขียนโปรแกรมเพื่อตรวจสอบผู้ใช้งานเข้าสู่ระบบเมื่อใดนั้น จำเป็นต้องมีแอปพลิเคชันที่ใช้สำหรับล็อกอินเอง กล่าวคือ ผู้ใช้จะต้องล็อกอินในแอปพลิเคชันนั่นเอง แต่โครงการนี้ ต้องการที่จะใช้ส่วนประกอบของวินโดวส์ 2000 ซึ่งมีระบบล็อกอินอยู่แล้ว จึงทำให้ไม่สามารถตรวจสอบผู้ใช้งานเข้าสู่ระบบเมื่อใดตามเวลาจริง โปรแกรมจะทำการตรวจสอบผู้ใช้ได้ก็ต่อเมื่อผู้ใช้เข้าสู่ระบบในระยะเวลาหนึ่ง ซึ่งอาจจะล่าช้าไปบ้าง แต่ก็ไม่ถึงกับเสียเวลามากมายนัก

ในส่วนของไอพีแอดเดรสนั้น วินโดวส์ 2000 ไม่มีฟังก์ชันที่จะสามารถรับค่าไอพีแอดเดรสจากเครื่องที่ผู้ใช้ใช้งาน เนื่องจากบนวินโดวส์ 2000 นั้นส่วนใหญ่จะใช้ชื่อผู้ใช้เป็นตัวอ้างอิงกับระบบ ดังนั้นจึงทำการตรวจสอบไอพีทุกไอพีภายในระบบ เพื่อให้ทราบว่าผู้ใช้ใดใช้เครื่องในไอพีใด หรือพูดอีกทางหนึ่งก็คือ โปรแกรมจะทำการตรวจสอบไอพีทุกไอพี โดยจะตรวจสอบว่าไอพีนั้นๆผู้ใช้ใดใช้งานอยู่บนระบบ แล้วจึงแจ้งให้กับทางไฟร์วอลล์ ซึ่งการตรวจสอบว่าผู้ใช้ใดใช้งานอยู่บนไอพีนั้นๆ ก็ทำได้โดยใช้เน็ตเอพียู (NetAPI Function) ซึ่งเป็นฟังก์ชันบริการ (Service Function) สำหรับนักพัฒนาที่วินโดวส์ 2000 ได้จัดเตรียมไว้

### 6.2 การเรียกใช้เน็ตเอพียูฟังก์ชัน

เน็ตเอพียู (Network Management Function) ซึ่งเป็นฟังก์ชันที่ใช้สำหรับจัดการกับบัญชีผู้ใช้และทรัพยากรสำหรับเน็ตเวิร์กสำหรับแอปพลิเคชันที่พัฒนาด้วยภาษา C และ C++ ซึ่งจะมีข้อดี ดังนี้

- เน็ตเอพียู จะมีฟังก์ชันที่เอพียู อื่นๆไม่สามารถทำได้
- เน็ตเอพียูใช้ได้ แม้ไม่มีแอคทีฟไคลเรกทอรี

ในส่วนของโปรแกรมได้ใช้เน็ตเอพียูเพื่อตรวจสอบว่าผู้ใช้ใดใช้งานอยู่บนไอพีแอดเดรสที่

กำลังตรวจสอบอยู่ สำหรับเน็ตเอพีไอ ที่ใช้นั้น ก็คือ NetWkstaUserEnum Function ซึ่งเป็นฟังก์ชันที่ใช้ตรวจสอบผู้ใช้ที่ใช้เครื่องนั้นๆอยู่ ซึ่งรายละเอียดของฟังก์ชันจะมี ดังนี้

```
NET_API_STATUS NetWkstaUserEnum(  
    LPWSTR servername,  
    DWORD level,  
    LPBYTE *bufptr,  
    DWORD prefmaxlen,  
    LPDWORD entriesread,  
    LPDWORD totalentries,  
    LPDWORD resumehandle  
);
```

**servername**

[in] Pointer to a string that specifies the DNS or NetBIOS name of the remote server on which the function is to execute. If this parameter is NULL, the local computer is used.

Windows NT 4.0 and earlier: This string must begin with \\.

**level**

[in] Specifies the information level of the data. This parameter can be one of the following values.

| Value | Meaning                                                                                                                                                     |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0     | Return the names of users currently logged on to the workstation. The bufptr parameter points to an array of WKSTA_user_INFO_0 structures.                  |
| 1     | Return the names of the current users and the domains accessed by the workstation. The bufptr parameter points to an array of WKSTA_user_INFO_1 structures. |

**bufptr**

[out] Pointer to the buffer that receives the data. The format of this data depends on the value of the level parameter. This buffer is allocated by the system and must be freed using the NetApiBufferFree function. Note that you must free the buffer even if the function fails with ERROR\_MORE\_DATA.

prefmaxlen

[in] Specifies the preferred maximum length of returned data, in bytes. If you specify MAX\_PREFERRED\_LENGTH, the function allocates the amount of memory required for the data. If you specify another value in this parameter, it can restrict the number of bytes that the function returns. If the buffer size is insufficient to hold all entries, the function returns ERROR\_MORE\_DATA. For more information, see Network Management Function Buffers and Network Management Function Buffer Lengths.

entriesread

[out] Pointer to a value that receives the count of elements actually enumerated.

totalentries

[out] Pointer to a value that receives the total number of entries that could have been enumerated from the current resume position.

resumehandle

[in/out] Pointer to a value that contains a resume handle which is used to continue an existing search. The handle should be zero on the first call and left unchanged for subsequent calls. If resumehandle is NULL, no resume handle is stored.

## Return Values

If the function succeeds, the return value is NERR\_Success.

If the function fails, the return value can be one of the following error codes.

| Value               | Meaning                                                                           |
|---------------------|-----------------------------------------------------------------------------------|
| ERROR_ACCESS_DENIED | The user does not have access to the requested information.                       |
| ERROR_MORE_DATA     | More entries are available. Specify a large enough buffer to receive all entries. |
| ERROR_INVALID_LEVEL | The level parameter is invalid.                                                   |

*รูปที่ 6-1 แสดงรายละเอียดของเน็ตเวิร์กพีไอ ที่เรียกใช้งาน*

จากรูป 6-1 จะแสดงรายละเอียดของฟังก์ชันเอพีไอที่เรียกใช้งานเพื่อตรวจสอบผู้ใช้ในแต่ละเครื่อง ซึ่งฟังก์ชันนี้ จะอยู่ในไฟล์ที่มีชื่อว่า getuserc.exe โดยจะทำการเอ็กซ์คิวต์ไฟล์จากโปรแกรมหลักของแอปพลิเคชัน ด้วยคำสั่ง `_spawnl` ซึ่งจะเป็นคำสั่งที่ใช้ในการเอ็กซ์คิวต์ไฟล์ โดยในส่วนของโปรแกรม `getuserc.exe` นั้น เมื่อทำการตรวจสอบผู้ใช้ของแต่ละเครื่องแล้ว จะใส่ชื่อผู้ใช้งานในเท็กซ์ไฟล์ (Text File) ที่มีชื่อว่า `user.txt`

เมื่อโปรแกรมหลักทำการเอ็กซ์คิวต์ด้วยคำสั่ง `_spawnl` แล้ว ก็จะมีการอ่านค่าจากไฟล์ `user.txt` แล้วจึงนำชื่อผู้ใช้งานมาประมวลผลต่อไป ดังแสดง

```
#ifndef UNICODE
#define UNICODE
#endif

#include <stdio.h>
#include <assert.h>
#include <fstream.h>
#include <windows.h>
#include <lm.h>
#include <process.h>
#pragma comment(lib,"netapi32")
#include <wchar.h>
#include <stdarg.h>

int wmain(int argc, wchar_t *argv[])
{
    LPWKSTA_USER_INFO_0 pBuf = NULL;
    LPWKSTA_USER_INFO_0 pTmpBuf=NULL;
    DWORD dwLevel = 0;
    DWORD dwPrefMaxLen = -1;
    DWORD dwEntriesRead = 0;
    DWORD dwTotalEntries = 0;
    DWORD dwResumeHandle = 0;
    DWORD i;
    DWORD dwTotalCount = 0;
    NET_API_STATUS nStatus;
    LPTSTR pszServerName = NULL;
```

```

FILE* stream;

if (argc > 2)
{
    fprintf(stderr, L"Usage: %s [\\ServerName]\n", argv[0]);
    exit(1);
}

if (argc == 2)
    pszServerName = argv[1];

do // begin do
{
    nStatus = NetWkstaUserEnum(pszServerName,
                               dwLevel,
                               (LPBYTE*)&pBuf,
                               dwPrefMaxLen,
                               &dwEntriesRead,
                               &dwTotalEntries,
                               &dwResumeHandle);

    if ((nStatus == NERR_Success) || (nStatus == ERROR_MORE_DATA))
    {
        if ((pTmpBuf = pBuf) != NULL)
        {
            for (i = 0; (i < dwEntriesRead); i++)
            {
                assert(pTmpBuf != NULL);

                if (pTmpBuf == NULL)
                {

                    fprintf(stderr, "An access violation has occurred\n");
                    break;
                }
            }
        }
    }
}

```

```

        pTmpBuf++;
        dwTotalCount++;
    }

    pTmpBuf--;
    stream = fopen( "vfwprintf.txt", "w" );
    fprintf( stream,"%S",pTmpBuf->wkui0_username);
    fclose( stream );

}
else
{
    stream = fopen( "vfwprintf.txt", "w" );
    fprintf( stream,"%S","");
    fclose( stream );
    exit(1);
}
}
else
{
    stream = fopen( "vfwprintf.txt", "w" );
    fprintf( stream,"%S","");
    fclose( stream );
    exit(1);
}

if (pBuf != NULL)
{
    NetApiBufferFree(pBuf);
    pBuf = NULL;
}

}while (nStatus == ERROR_MORE_DATA); // end do

```

```

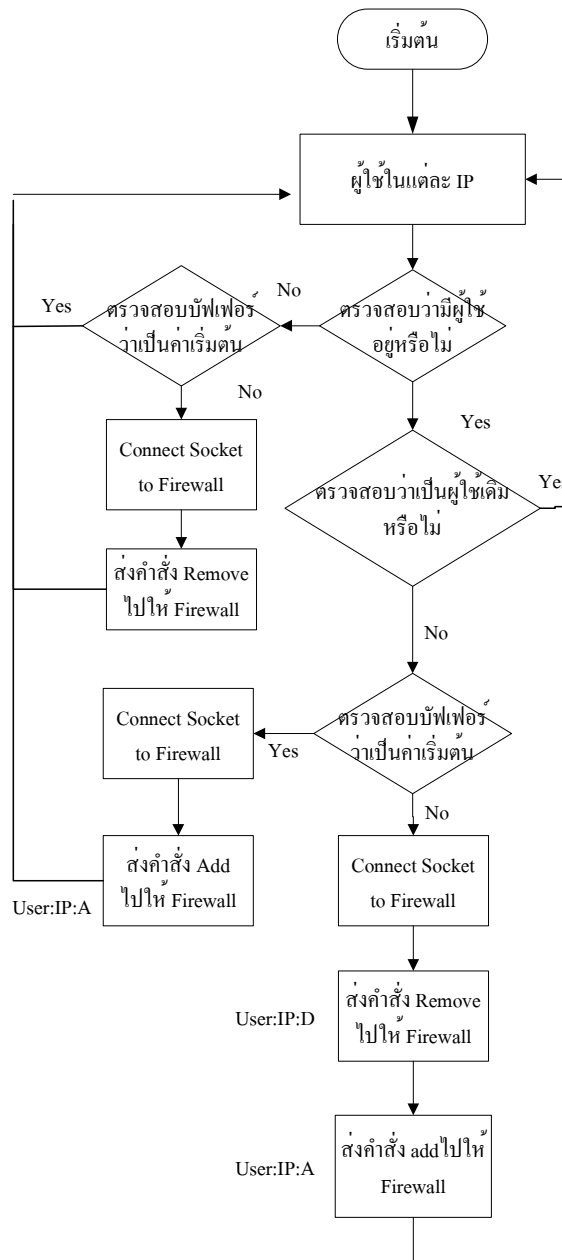
if (pBuf != NULL)
    NetApiBufferFree(pBuf);
return 0;
}

```

**รูป 6-2 แสดงโค้ดของไฟล์ *userc.exe***

### **6.3 การตรวจสอบการเข้าออกของผู้ใช้**

สำหรับการตรวจสอบการเข้าออกของผู้ใช้นั้น เราได้ใช้ฟังก์ชัน NetAPI ดังที่ได้กล่าวมาในหัวข้อที่ผ่านมา โดยจะทำการตรวจสอบว่าผู้ใช้ใดใช้งานอยู่บนเครื่องใด แต่ในการตรวจสอบว่าผู้ใช้ใดออกจากระบบนั้น จะใช้ฟังก์ชันนี้เช่นกันในการตรวจสอบ โดยโปรแกรมเมื่อทำการตรวจสอบถึงไอพีใด ก็จะตรวจสอบว่าผู้ใช้ที่ใช้งานอยู่ยังเป็นผู้ใช้เดิมหรือไม่ หากเป็นผู้ใช้เดิมก็จะเข้าไปตรวจสอบไอพีอื่น แต่หากผู้ใช้เกิดการเปลี่ยนแปลงขึ้น ก็จะทำการส่งข้อมูลของผู้ใช้เก่า ซึ่งถูกเก็บในบัฟเฟอร์ให้กับไฟร์วอลล์ว่า ผู้ใช้เก่าได้ออกจากระบบไปแล้ว จากนั้นจะนำผู้ใช้ใหม่ส่งให้กับไฟร์วอลล์ทันทีและเก็บผู้ใช้ใหม่ในบัฟเฟอร์แทนที่ผู้ใช้เก่านั้น ดังรูป 6-3 จะเป็นโฟลว์ชาร์ต (Flow Chart) ของโปรแกรม



รูปที่ 6-3 แสดงฟลัวชาร์ต (Flow Chart) ของการทำงานในส่วนการตรวจสอบการเข้าออกของผู้ใช้

#### 6.4 การติดต่อกับไฟร์วอลล์ผ่านซ็อกเก็ต (Socket)

ในการที่จะทำให้ไฟร์วอลล์ทราบว่ามีผู้ใช้ใดเข้าสู่ระบบและได้ใช้เครื่องใดนั้น แอปพลิเคชันฝั่งวินโดวส์ 2000 จะต้องทำการส่งข้อมูลไปให้ ซึ่งบนฝั่งไฟร์วอลล์เองก็จะมีโมดูลสำหรับรับข้อมูลนั้นอยู่แล้ว โดยการจะส่งไปได้นั้น ก็จะใช้การส่งข้อมูลผ่านทางซ็อกเก็ต ซึ่งการเขียนโปรแกรมผ่านซ็อกเก็ตนั้น เป็นการเขียนโปรแกรมทางเน็ตเวิร์ก ที่ใช้กันอย่างแพร่หลาย ดังนั้นโครงงานนี้จึงได้เลือกการเขียนโปรแกรมด้วยวิธีนี้มาพัฒนาแอปพลิเคชันขึ้น ซึ่งในการเขียนโปรแกรมผ่านทางซ็อกเก็ตนั้นจะต้องอ้างอิงกับพอร์ต



ด้วยว่าใช้พอร์ตใดในการติดต่อ ซึ่งในโปรแกรมนี้จะใช้งานพอร์ต 1170 ซึ่งเป็นพอร์ตที่ไม่ได้ถูกกำหนดให้ใช้งานอย่างตายตัว มาใช้ในการติดต่อกันระหว่างไฟร์วอลล์และแอปพลิเคชันบนวินโดวส์ 2000

#### 6.4.1 การเขียนโปรแกรมผ่านทางซ็อกเก็ต

ในการเขียนโปรแกรมทางเน็ตเวิร์ก การเขียนโปรแกรมโดยใช้ซ็อกเก็ตนั้น เป็นที่นิยมอย่างแพร่หลาย และถือเป็นพื้นฐานของการเขียนโปรแกรมทางเน็ตเวิร์ก

สำหรับการเขียนโปรแกรมโดยใช้ซ็อกเก็ตบนวินโดวส์ 2000 นั้น จะคล้ายๆกับการเขียนโปรแกรมผ่านทางซ็อกเก็ตบนลินุกซ์ โดยจะแตกต่างกันเพียง ในการเขียนโปรแกรมบนวินโดวส์นั้น จะต้องเรียกใช้คำสั่ง WSASStartup เพื่อเป็นการเริ่มต้นการเรียกใช้งาน DLL บนวินโดวส์ด้วย โดยมีรูปแบบดังนี้

Int WSASStartup(

Word wVersionRequired,  
LPWSADATA lpWSADATA);

- wVersionRequired จะเป็นค่าของเวอร์ชัน DLL ที่เรียกใช้
- LPWSADATA จะเป็นพอยน์เตอร์ไปยัง WSADATA Structure ซึ่งจะเป็นส่วนที่แสดงรายละเอียดของ DLL

เมื่อทำการกำหนดค่าเริ่มต้นให้กับ DLL แล้ว ก็จะต้อง เปิดซ็อกเก็ต (Open Socket) สำหรับใช้ในการติดต่อ โดยจะมีรูปแบบคำสั่งดังนี้

Typedef unsigned int SOCKET;

SOCKET PASCAL FAR socket(int af,int type, int protocol);

- af แสดงค่าของ แอดเดรสแฟมิลี่
- type ชนิดการส่งข้อมูลเป็น Datagram หรือ Stream
- protocol ชนิดของโปรโตคอล เป็น TCP หรือ UDP

เมื่อทำการเปิดซ็อกเก็ตแล้ว ก็จะทำการ listen หรือ connect ไป แล้วแต่การติดต่อ หากเป็นฝั่งเซิร์ฟเวอร์ ก็จะทำการ listen และเมื่อมีการ connect จากฝั่งไคลเอ็นท์ก็จะ accept หากเป็น ฝั่งไคลเอ็นท์ร้องขอการติดต่อก็จะ connect

#### รูปแบบคำสั่ง listen

Int PASCAL listen (SOCKET s,int backlog);

### รูปแบบคำสั่ง **accept**

SOCKET PASCAL accept(SOCKET s, LPSOCKADDR lpAddr , LPINT lpLength);

### รูปแบบคำสั่ง **connect**

Int PASCAL connect(SOCKET s,const struct sockaddr \*name, int namelen);

เมื่อทำการติดต่อกันระหว่างเครื่องไคลเอนท์กับเซิร์ฟเวอร์แล้ว ก็จะทำการส่งข้อมูลกันไปมาระหว่างไคลเอนท์กับเซิร์ฟเวอร์โดยผ่านทางซ็อกเก็ตที่ได้กำหนดไว้

### รูปแบบคำสั่ง **send**

Int PASCAL send(SOCKET s, LPCSTR buf, int len, int flags);

### รูปแบบคำสั่ง **receive**

Int PASCAL recv(SOCKET s, LPSTR buf , int len , int flags);

เมื่อทำการติดต่อบริ่ส่งข้อมูลกันเสร็จเรียบร้อยแล้ว ก็ควรจะปิดซ็อกเก็ตนั้นทิ้งไป เพื่อป้องกันการจู่โจมจากผู้ไม่ประสงค์ดี ด้วยคำสั่ง ดังนี้

### รูปแบบคำสั่ง **close**

Int PASCAL closesocket(SOCKET s);

ดังที่กล่าวมาแล้วว่าในการเขียนโปรแกรมบนวินโดวส์นั้น จะต้องใช้ DLL และในตอนเริ่มต้นได้มีการกำหนดค่า DLL ที่ใช้งาน ดังนั้นเมื่อเลิกใช้งานซ็อกเก็ตก็ต้องปิด DLL ที่ใช้ด้วย ด้วยคำสั่ง WSACleanUp();

### ตัวอย่างการเขียนโปรแกรม ทางเน็ตเวิร์กโดยผ่านทางซ็อกเก็ต

```
WSADATA wsData;  
  
WORD wVer = MAKEWORD(2,2);  
  
SOCKET minix;  
  
char *szName=MYHOST;  
  
int status;  
  
struct hostent *he;
```

```

WCHAR  wszRecvBuf[100];

struct sockaddr_in their_addr;

memset(wszRecvBuf,0,sizeof(wszRecvBuf));

status = WSASStartup(wVer,&wsData);

minix=socket(AF_INET,SOCK_STREAM,0);

if (minix == INVALID_SOCKET)

{

    printf("Failed to create socket: %d\n",WSAGetLastError());

    WSACleanup();

    return -1;

}

bind(minix,(const sockaddr *)&their_addr,sizeof(their_addr));

their_addr.sin_family=AF_INET;

their_addr.sin_port=htons(MYPORT);

int len=0;

gethostname(szName,len);

he = gethostbyname(szName);

their_addr.sin_addr.S_un.S_addr = *((long *) (he->h_addr));

memset(&(their_addr.sin_zero),'\0',8);

status= connect(minix,(struct sockaddr*) &their_addr,sizeof(struct sockaddr)) ;

if(status==-1)

{

    printf("\nfail to connect");

    WSAGetLastError();

    WSACleanup();

    return -1;

}

char* msg2 = (char*) malloc (sizeof(char*));

status = send(minix,(char*)msg,100,0);

printf("\nsend %s:",msg);

if(status<0)

{

    printf("\nfail to send");

```

```

WSAGetLastError();

WSACleanup();

}

```

#### 6.4.2 รูปแบบข้อมูลที่ใช้ในการติดต่อระหว่างแอปพลิเคชันกับไฟร์วอลล์

ในการส่งข้อมูลนั้น เมื่อโปรแกรมตรวจสอบพบว่าผู้ใช้เข้าสู่ระบบบนไอพีใด ก็จะนำ ชื่อผู้ใช้งานร่วมกับไอพีแอดเดรสโดยขึ้นด้วยเครื่องหมาย ‘.’ จากนั้นก็จะส่งให้กับไฟร์วอลล์ แต่การที่จะบอกไฟร์วอลล์ว่าข้อมูลที่ส่งไปนั้นเป็นการเข้าหรือออกของผู้ใช้นั้น โปรแกรมก็จะทำการรวมตัวอักษรไว้ต่อท้ายเพื่อให้ไฟร์วอลล์ทราบว่าเป็นการเข้าหรือการออกของผู้ใช้ ซึ่งหากต่อท้ายด้วย A ก็จะหมายถึงผู้ใช้เข้าสู่ระบบ หากต่อท้ายด้วยตัว D ก็จะหมายถึงผู้ใช้ออกจากระบบนั่นเอง เช่น

|                    |         |                                                |
|--------------------|---------|------------------------------------------------|
| userA:192.168.0.5D | หมายถึง | ผู้ใช้ชื่อ userA ออกจากระบบบนไอพี 192.168.0.5  |
| userB:192.168.0.6A | หมายถึง | ผู้ใช้ชื่อ userB เข้าสู่ระบบบนไอพี 192.168.0.6 |

ซึ่งข้อมูลที่ถูกส่งไปนี้ จะอยู่รูปแบบสตริง ซึ่งแอปพลิเคชันบนฝั่งไฟร์วอลล์จะทำการตัดคำออกมาโดยใช้เครื่องหมาย ‘.’ นี้เพื่อแยกแยะระหว่างชื่อผู้ใช้งานกับไอพีแอดเดรส และสุดท้าย ตัว D หรือ A ออกมาเพื่อทำการเพิ่มโปรเซสหรือลบโปรเซสนั่นเอง